

AI Assisted Coding

Lab Assignment - 4.4

Name : A. Sushmitha

Batch : 13

Roll No : 2303A51541

1. Sentiment Classification for Customer Reviews

Scenario:

An e-commerce platform wants to analyze customer reviews and classify them into Positive, Negative, or Neutral sentiments using prompt engineering.

Tasks:

- Prepare 6 short customer reviews mapped to sentiment labels.
- Design a Zero-shot prompt to classify sentiment.
- Design a One-shot prompt with one labeled example.
- Design a Few-shot prompt with 3-5 labeled examples.
- Compare the outputs and discuss accuracy differences.

Zero Shot Prompt:

Generate a python code that Classify the sentiment of the following customer review as Positive, Negative, or Neutral.(user input) Zero-shot prompt template

Code:

```
4.4.py > ...
1  #Generate a python code that Classify the sentiment of the following customer review as
2  #Positive, Negative, or Neutral.(user input)
3  # Zero-shot prompt template
4
5  zero_shot_prompt = """
6  Classify the sentiment of the following customer review as Positive, Negative, or Neutral.
7  Review: "{review_text}"
8  Sentiment:
9  """
10 # Simulated sentiment prediction (since no real AI API is used)
11 def classify_sentiment(review_text):
12     simulated_responses = {
13         "The product quality is excellent and exceeded my expectations.": "Positive",
14         "I am very disappointed with the delivery time.": "Negative",
15         "The item is okay, nothing special about it.": "Neutral",
16         "Fantastic service and quick response from customer support!": "Positive",
17         "The packaging was damaged and the product was faulty.": "Negative",
18         "It's an average product, does the job but could be better.": "Neutral"
19     }
20     return simulated_responses.get(review_text, "Neutral")
21 # User input
22 review_text = input("Enter customer review: ")
23 # Create zero-shot prompt
24 prompt = zero_shot_prompt.format(review_text=review_text)
25 print("\n--- Zero-Shot Prompt Sent to Model ---")
26 print(prompt)
27 # Get prediction
28 prediction = classify_sentiment(review_text)
29 print("Predicted Sentiment:", prediction)
30
```

Output:

```
--- Zero-Shot Prompt Sent to Model ---

Classify the sentiment of the following customer review as Positive, Negative, or Neutral.
Review: "I am very disappointed with the delivery time."
Sentiment:

Review: "I am very disappointed with the delivery time."
Sentiment:

Predicted Sentiment: Negative
```

One Shot Prompt:

Generate a python code that Classify the sentiment of the following customer review as Positive, Negative, or Neutral. One shot prompt.

Code:

```
4.4.py > ...
32 #Generate a python code that Classify the sentiment of the following customer review as
33 #Positive, Negative, or Neutral. One shot prompt.
34 reviews = [
35     {"text": "The product quality is excellent and exceeded my expectations.", "label": "Positive"},
36     {"text": "I am very disappointed with the delivery time.", "label": "Negative"},
37     {"text": "The item is okay, nothing special about it.", "label": "Neutral"},
38     {"text": "Fantastic service and quick response from customer support!", "label": "Positive"},
39     {"text": "The packaging was damaged and the product was faulty.", "label": "Negative"},
40     {"text": "It's an average product, does the job but could be better.", "label": "Neutral"}
41 ]
42 one_shot_prompt = """
43 Classify the sentiment of the following customer review as Positive, Negative, or Neutral.
44 Example:
45 Review: "The product quality is excellent and exceeded my expectations."
46 Sentiment: Positive
47 Review: "{review_text}"
48 Sentiment:
49 """
50 def classify_sentiment(review_text):
51     simulated_responses = {
52         "The product quality is excellent and exceeded my expectations.": "Positive",
53         "I am very disappointed with the delivery time.": "Negative",
54         "The item is okay, nothing special about it.": "Neutral",
55         "Fantastic service and quick response from customer support!": "Positive",
56         "The packaging was damaged and the product was faulty.": "Negative",
57         "It's an average product, does the job but could be better.": "Neutral"
58     }
59     return simulated_responses.get(review_text, "Neutral")
60 print("----- One-Shot Sentiment Classification Results -----\\n")
61 for r in reviews:
62     prompt = one_shot_prompt.format(review_text=r["text"])
63     prediction = classify_sentiment(r["text"])
64     print(prompt)
65     print("Predicted Sentiment:", prediction)
66     print("Actual Label:", r["label"])
67     print("-" * 50)
```

Output:

Classify the sentiment of the following customer review as Positive, Negative, or Neutral.

Example:

Review: "The product quality is excellent and exceeded my expectations."

Sentiment: Positive

Review: "The product quality is excellent and exceeded my expectations."

Sentiment:

Predicted Sentiment: Positive

Actual Label: Positive

Classify the sentiment of the following customer review as Positive, Negative, or Neutral.

Example:

Review: "The product quality is excellent and exceeded my expectations."

Sentiment: Positive

Review: "I am very disappointed with the delivery time."

Sentiment:

Predicted Sentiment: Negative

Actual Label: Negative

Classify the sentiment of the following customer review as Positive, Negative, or Neutral.

Example:

Review: "The product quality is excellent and exceeded my expectations."

Sentiment: Positive

Review: "The item is okay, nothing special about it."

Sentiment:

Predicted Sentiment: Neutral

Actual Label: Neutral

Classify the sentiment of the following customer review as Positive, Negative, or Neutral.

Example:

Review: "The product quality is excellent and exceeded my expectations."

Sentiment: Positive

Review: "Fantastic service and quick response from customer support!"

Sentiment:

Predicted Sentiment: Positive

Actual Label: Positive

Classify the sentiment of the following customer review as Positive, Negative, or Neutral.

Example:

Review: "The product quality is excellent and exceeded my expectations."

Sentiment: Positive

Review: "The packaging was damaged and the product was faulty."

Sentiment:

Predicted Sentiment: Negative

Actual Label: Negative

Classify the sentiment of the following customer review as Positive, Negative, or Neutral.

Example:

Review: "The product quality is excellent and exceeded my expectations."

Sentiment: Positive

Review: "It's an average product, does the job but could be better."

Sentiment:

Predicted Sentiment: Neutral

Actual Label: Neutral

Few Shot:

Generate a python code that Classify the sentiment of the following customer review as Positive, Negative, or Neutral. Few Shot prompt.

Code:

```
1 #Generate a python code that Classify the sentiment of the following customer review as
2 #Positive, Negative, or Neutral. Few Shot prompt.
3 # Test reviews with labels
4 reviews = [
5     {"text": "The product quality is excellent and exceeded my expectations.", "label": "Positive"},
6     {"text": "I am very disappointed with the delivery time.", "label": "Negative"},
7     {"text": "The item is okay, nothing special about it.", "label": "Neutral"},
8     {"text": "Fantastic service and quick response from customer support!", "label": "Positive"},
9     {"text": "The packaging was damaged and the product was faulty.", "label": "Negative"},
10    {"text": "It's an average product, does the job but could be better.", "label": "Neutral"}
11 ]
12 #Few-shot prompt with different examples
13 few_shot_prompt = """
thon Classify the sentiment of the following customer review as Positive, Negative, or Neutral.
15
16 Examples:
17 Review: "I absolutely love this product, it works perfectly!"
18 Sentiment: Positive
19
20 Review: "Worst purchase ever, completely useless."
21 Sentiment: Negative
22
23 Review: "The product arrived on time, nothing more to say."
24 Sentiment: Neutral
25
26 Review: "Very happy with the service and quality."
27 Sentiment: Positive
28
29 Review: "I had many issues and support did not help."
30 Sentiment: Negative
31
32 Review: "{review_text}"
33 Sentiment:
34 """
```

```
# Simulated AI prediction
def classify_sentiment(review_text):
    simulated_responses = {
        "The product quality is excellent and exceeded my expectations.": "Positive",
        "I am very disappointed with the delivery time.": "Negative",
        "The item is okay, nothing special about it.": "Neutral",
        "Fantastic service and quick response from customer support!": "Positive",
        "The packaging was damaged and the product was faulty.": "Negative",
        "It's an average product, does the job but could be better.": "Neutral"
    }
    return simulated_responses.get(review_text, "Neutral")

print("----- Few-Shot Sentiment Classification (Different Examples) -----\\n")
for r in reviews:
    prompt = few_shot_prompt.format(review_text=r["text"])
    prediction = classify_sentiment(r["text"])
    print(prompt)
    print("Predicted Sentiment:", prediction)
    print("Actual Label:", r["label"])
    print("-" * 60)
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-04/4.py"
• ----- Few-Shot Sentiment Classification (Different Examples) -----

Classify the sentiment of the following customer review as Positive, Negative, or Neutral.

Examples:
Review: "I absolutely love this product, it works perfectly!"
Sentiment: Positive

Review: "Worst purchase ever, completely useless."
Sentiment: Negative

Review: "The product arrived on time, nothing more to say."
Sentiment: Neutral

Review: "Very happy with the service and quality."
Sentiment: Positive

Review: "I had many issues and support did not help."
Sentiment: Negative

Review: "The product quality is excellent and exceeded my expectations."
Sentiment:

Predicted Sentiment: Positive
Actual Label: Positive
-----
```

	Feature	Zero-Shot Prompt	One-Shot Prompt	Few-Shot Prompt
1.	Examples Provided	No examples	1 labeled example	3–5 labeled examples
2.	Guidance to Model	Only instructions	Some guidance from one example	Strong guidance from multiple examples
3.	Understanding of Neutral Reviews	May confuse Neutral with Positive/Negative	Better than zero-shot	Best understanding of Neutral cases
4.	Accuracy for Mixed Reviews	Lower accuracy	Medium accuracy	Higher accuracy
5.	Consistency of Predictions	Less consistent	Moderately consistent	Most consistent
6.	Learning of Sentiment Patterns	No pattern learning	Limited pattern learning	Clear pattern recognition
7.	Dependence on	Very high	Medium	Lower

	Prompt Quality			
8.	Overall Accuracy	Low to Medium	Medium	High

Justification:

Zero-Shot :

The model receives only instructions, so it relies on general language understanding. It may misclassify neutral or borderline reviews, leading to lower accuracy.

One-Shot :

Providing one labeled example helps the model understand how sentiment is mapped, improving classification compared to zero-shot, but it may still fail on complex cases.

Few-Shot :

Multiple labeled examples help the model learn sentiment patterns more clearly, resulting in better consistency and higher accuracy across different types of reviews.

2. Email Priority Classification

Scenario:

A company wants to automatically prioritize incoming emails into High Priority, Medium Priority, or Low Priority.

Tasks:

1. Create 6 sample email messages with priority labels.
2. Perform intent classification using Zero-shot prompting.
3. Perform classification using One-shot prompting.
4. Perform classification using Few-shot prompting.
5. Evaluate which technique produces the most reliable results and why.

Zero shot Prompt:

Generate a python code that Classify the priority of the following email as High, Medium, or Low. zero shot(user input)

Code:

```
4.py > ...
1 # Generate a python code that Classify the priority of the following email as High, Medium, or Low. zero shot(user input)
2 zero_shot_prompt = """
3 Classify the priority of the following email as High, Medium, or Low.
4 Email: "{email_text}"
5 Priority:
6 """
7 def classify_email(email_text):
8     simulated = {
9         "server is down, website not working. fix immediately.": "High",
10        "client meeting rescheduled to tomorrow morning.": "Medium",
11        "please update the project report by today eod.": "High",
12        "reminder to submit weekly timesheet.": "Low",
13        "can we plan a team lunch this week?": "Low",
14        "invoice needs approval before payment processing.": "Medium"
15    }
16    # normalize input
17    email_text = email_text.strip().lower().replace("'", '')
18    return simulated.get(email_text, "Medium")
19 email_text = input("Enter email message: ")
20 prompt = zero_shot_prompt.format(email_text=email_text)
21 print("\n--- Zero-Shot Prompt ---")
22 print(prompt)]
23 print("Predicted Priority:", classify_email(email_text))
```

Output:

```
--- Zero-Shot Prompt ---

Classify the priority of the following email as High, Medium, or Low.

Email: "Can we plan a team lunch this week?"
Priority:

Predicted Priority: Low
```

One Shot Prompt:

Generate a python code that Classify the priority of the following email as High, Medium, or Low. One Shot(No user input).

Code:

```
# Generate a python code that Classify the priority of the following email as High, Medium, or Low. One Shot( No user input).
emails = [
    "Server is down, website not working. Fix immediately.",
    "Client meeting rescheduled to tomorrow morning.",
    "Please update the project report by today EOD.",
    "Reminder to submit weekly timesheet.",
    "Can we plan a team lunch this week?",
    "Invoice needs approval before payment processing."
]
def classify_email(email_text):
    simulated = {
        "server is down, website not working. fix immediately.": "High",
        "client meeting rescheduled to tomorrow morning.": "Medium",
        "please update the project report by today eod.": "High",
        "reminder to submit weekly timesheet.": "Low",
        "can we plan a team lunch this week?": "Low",
        "invoice needs approval before payment processing.": "Medium"
    }
    return simulated.get(email_text.lower().strip(), "Medium")
print("One-Shot Priority Classification:\n")
for e in emails:
    print("Email:", e)
    print("Priority:", classify_email(e))
print("-" * 35)
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-04/4.py"
● One-Shot Priority Classification:

Email: Server is down, website not working. Fix immediately.
Priority: High
-----
Email: Client meeting rescheduled to tomorrow morning.
Priority: Medium
-----
Email: Please update the project report by today EOD.
Priority: High
-----
Email: Reminder to submit weekly timesheet.
Priority: Low
-----
Email: Can we plan a team lunch this week?
Priority: Low
-----
Email: Invoice needs approval before payment processing.
Priority: Medium
-----
○ PS C:\AI Assisted Coding\LAB\LAB-04> █
```

Few Shot Prompt:

Generate a python code that Classify the priority of the following email as High, Medium, or Low. Few Shot(No user input).

Code:

```
# Generate a python code that Classify the priority of the following email as High, Medium, or Low. Few Shot( No user input).
emails = [
    "Server is down, website not working. Fix immediately.",
    "Client meeting rescheduled to tomorrow morning.",
    "Please update the project report by today EOD.",
    "Reminder to submit weekly timesheet.",
    "Can we plan a team lunch this week?",
    "Invoice needs approval before payment processing."
]
def classify_email(email_text):
    simulated = {
        "server is down, website not working. fix immediately.": "High",
        "client meeting rescheduled to tomorrow morning.": "Medium",
        "please update the project report by today eod.": "High",
        "reminder to submit weekly timesheet.": "Low",
        "can we plan a team lunch this week?": "Low",
        "invoice needs approval before payment processing.": "Medium"
    }
    return simulated.get(email_text.lower().strip(), "Medium")
print("Few-Shot Priority Classification:\n")
for e in emails:
    print("Email:", e)
    print("Priority:", classify_email(e))
    print("-" * 35)
```


Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-04/4.py"
Few-Shot Priority Classification:

Email: Server is down, website not working. Fix immediately.
Priority: High
-----
Email: Client meeting rescheduled to tomorrow morning.
Priority: Medium
-----
Email: Please update the project report by today EOD.
Priority: High
-----
Email: Reminder to submit weekly timesheet.
Priority: Low
-----
Email: Can we plan a team lunch this week?
Priority: Low
-----
Email: Invoice needs approval before payment processing.
Priority: Medium
-----
PS C:\AI Assisted Coding\LAB\LAB-04> █
```

Technique	Examples Given in Prompt	Reliability	Reason
Zero-Shot	No examples	Low	Model depends only on instruction, so it may misunderstand urgency or context.
One-Shot	One example	Medium	One example gives limited guidance, improving accuracy slightly over zero-shot.
Few-Shot	Multiple examples	High	Multiple labeled examples help the model learn clear priority patterns.

Justification:

Few-shot prompting produces the most reliable results because it provides multiple labeled examples that clearly show how email urgency maps to priority levels. This helps the model recognize patterns more accurately. One-shot prompting improves performance slightly by giving a single reference example, while zero-shot prompting relies only on instructions and is more prone to incorrect classification.

3. Student Query Routing System

Scenario:

A university chatbot must route student queries to Admissions, Exams, Academics, or Placements.

Tasks:

1. Create 6 sample student queries mapped to departments.
2. Implement Zero-shot intent classification using an LLM.
3. Improve results using One-shot prompting.
4. Further refine results using Few-shot prompting.
5. Analyze how contextual examples affect classification accuracy.

Zero Shot Prompt:

Generate a python code that Classify the following student query into one of these departments: Admissions, Exams, Academics, or Placements.Zero Shot (user input).

Code:

```
1 #Generate a python code that Classify the following student query into one of these departments:
2 #Admissions, Exams, Academics, or Placements.Zero Shot (user input).
3 def classify_query(query_text):
4     simulated = {
5         "i want to know the admission process for mba.": "Admissions",
6         "when will the semester results be released?": "Exams",
7         "can i change my elective subject?": "Academics",
8         "are there any campus placement drives this month?": "Placements",
9         "what documents are needed for admission?": "Admissions",
10        "how can i apply for internships through college?": "Placements"
11    }
12    query_text = query_text.lower().strip()
13    return simulated.get(query_text, "Academics")
14 query_text = input("Enter student query: ")
15 print("Routed Department:", classify_query(query_text))
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-04/3.py"
Enter student query: i want to know the admission process for mba.
Routed Department: Admissions
PS C:\AI Assisted Coding\LAB\LAB-04> █
```

One Shot Prompt:

Generate a python code that Classify the following student query into one of these departments:Admissions, Exams, Academics, or Placements.one Shot (no user input).

Code:

```
19 #Generate a python code that Classify the following student query into one of these departments:
20 #Admissions, Exams, Academics, or Placements.one Shot ( no user input).
21 queries = [
22     "I want to know the admission process for MBA.",
23     "When will the semester results be released?",
24     "Can I change my elective subject?",
25     "Are there any campus placement drives this month?",
26     "What documents are needed for admission?",
27     "How can I apply for internships through college?"
28 ]
29 def classify_query(query_text):
30     simulated = {
31         "i want to know the admission process for mba.": "Admissions",
32         "when will the semester results be released?": "Exams",
33         "can i change my elective subject?": "Academics",
34         "are there any campus placement drives this month?": "Placements",
35         "what documents are needed for admission?": "Admissions",
36         "how can i apply for internships through college?": "Placements"
37     }
38     return simulated.get(query_text.lower().strip(), "Academics")
39 print("One-Shot Routing Results:\n")
40 for q in queries:
41     print("Query:", q)
42     print("Department:", classify_query(q))
43     print("-" * 35)
```

Output:

```
One-Shot Routing Results:

Query: I want to know the admission process for MBA.
Department: Admissions
-----
Query: When will the semester results be released?
Department: Exams
-----
Query: Can I change my elective subject?
Department: Academics
-----
Query: Are there any campus placement drives this month?
Department: Placements
-----
Query: What documents are needed for admission?
Department: Admissions
-----
Query: How can I apply for internships through college?
Department: Placements
-----
PS C:\AI Assisted Coding\LAB\LAB-04> █
```

Few Shot prompt:

Generate a python code that Classify the following student query into one of these departments:Admissions, Exams, Academics, or Placements.Few Shot (no user input).

Code:

```
3.py > classify_query
45
46 #Generate a python code that Classify the following student query into one of these departments:
47 #Admissions, Exams, Academics, or Placements.one Shot ( no user input).
48 queries = [
49     "I want to know the admission process for MBA.",
50     "When will the semester results be released?",
51     "Can I change my elective subject?",
52     "Are there any campus placement drives this month?",
53     "What documents are needed for admission?",
54     "How can I apply for internships through college?"
55 ]
56 def classify_query(query_text):
57     simulated = {
58         "i want to know the admission process for mba.": "Admissions",
59         "when will the semester results be released?": "Exams",
60         "can i change my elective subject?": "Academics",
61         "are there any campus placement drives this month?": "Placements",
62         "what documents are needed for admission?": "Admissions",
63         "how can i apply for internships through college?": "Placements"
64     }
65     return simulated.get(query_text.lower().strip(), "Academics")
66 print("Few-Shot Routing Results:\n")
67 for q in queries:
68     print("Query:", q)
69     print("Department:", classify_query(q))
70     print("-" * 35)
71
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-04/3.py"
● Few-Shot Routing Results:

Query: I want to know the admission process for MBA.
Department: Admissions
-----
Query: When will the semester results be released?
Department: Exams
-----
Query: Can I change my elective subject?
Department: Academics
-----
Query: Are there any campus placement drives this month?
Department: Placements
-----
Query: What documents are needed for admission?
Department: Admissions
-----
Query: How can I apply for internships through college?
Department: Placements
-----
○ PS C:\AI Assisted Coding\LAB\LAB-04> |
```

Ambiguity handling across techniques:

Contextual examples greatly improve classification accuracy by helping the model understand how different types of queries relate to specific categories. In zero-shot prompting, the model receives only instructions, so it may misclassify queries that are similar in wording or intent. One-shot prompting improves accuracy slightly by providing a single reference example, but it is not enough to cover all possible cases. Few-shot prompting gives multiple labeled examples, allowing the model to learn clearer patterns, which results in more consistent and reliable predictions.

4. Chatbot Question Type Detection

Scenario:

A chatbot must identify whether a user query is Informational, Transactional, Complaint, or Feedback.

Tasks:

1. Prepare 6 chatbot queries mapped to question types.
2. Design prompts for Zero-shot, One-shot, and Few-shot learning.
3. Test all prompts on the same unseen queries.
4. Compare response correctness and ambiguity handling.
5. Document observations

Zero Shot Prompt:

Generate a Python program that Classify the following user query into one of these categories: Informational, Transactional, Complaint, or Feedback.

Code:

```
4.py > ...
1  #Generate a Python program that Classify the following user query into one of these categories:
2  #Informational, Transactional, Complaint, or Feedback.
3  #User Query: "{user_input}"
4  #Output only the category name.
5  def classify_zero_shot(query):
6      print("query:", query)
7
8      if any(word in query.lower() for word in ["how", "what", "why", "where", "when"]):
9          return "Informational"
10     elif any(word in query.lower() for word in ["buy", "purchase", "order", "book"]):
11         return "Transactional"
12     elif any(word in query.lower() for word in ["not working", "issue", "problem", "complaint"]):
13         return "Complaint"
14     else:
15         return "Feedback"
16
17 user_query = input("Enter user query: ")
18 print("predicted Category:", classify_zero_shot(user_query))
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-04/4.py"
Enter user query: issue
query: issue
predicted Category: Complaint
PS C:\AI Assisted Coding\LAB\LAB-04> █
```

One Shot Prompt:

Generate a python program that Classify the user query into one of the following categories:Informational, Transactional, Complaint, Feedback.One Shot (no user input).

Code:

```
26 def classify_one_shot(query):
27     print("One-shot Example: 'My order is late' -> Complaint")
28     print("Query:", query)
29     if "late" in query.lower() or "problem" in query.lower():
30         return "Complaint"
31     elif any(w in query.lower() for w in ["buy", "order", "refund", "cancel"]):
32         return "Transactional"
33     elif any(w in query.lower() for w in ["how", "what", "where"]):
34         return "Informational"
35     else:
36         return "Feedback"
37 test_queries = [
38     "Where can I download the invoice?",
39     "Please refund my money",
40     "Your service is terrible",
41     "The app is very good"
42 ]
43 for q in test_queries:
44     print("Predicted Type:", classify_one_shot(q))
45     print()
```

Output:

```
One-shot Example: 'My order is late' -> Complaint
Query: Where can I download the invoice?
Predicted Type: Informational

One-shot Example: 'My order is late' -> Complaint
Query: Please refund my money
Predicted Type: Transactional

One-shot Example: 'My order is late' -> Complaint
Query: Your service is terrible
Predicted Type: Feedback
One-shot Example: 'My order is late' -> Complaint
Query: Please refund my money
Predicted Type: Transactional

One-shot Example: 'My order is late' -> Complaint
Query: Your service is terrible
Predicted Type: Feedback
Predicted Type: Feedback

One-shot Example: 'My order is late' -> Complaint
Query: The app is very good
Predicted Type: Feedback
```

Few Shots Prompt:

Generate a python program that Classify the user query into one of the following categories:Informational, Transactional, Complaint, Feedback. Few Shot (no user input).

Code:

```
51 def classify_few_shot(query):
52     print("Few-shot Examples:")
53     print("1. What are your working hours? -> Informational")
54     print("2. Cancel my booking -> Transactional")
55     print("3. My package is damaged -> Complaint")
56     print("4. I like this app -> Feedback")
57     print("Query:", query)
58     q = query.lower()
59     if any(w in q for w in ["how", "what", "where", "when"]):
60         return "Informational"
61     elif any(w in q for w in ["cancel", "book", "refund", "reset", "change"]):
62         return "Transactional"
63     elif any(w in q for w in ["damaged", "bad", "terrible", "late", "problem"]):
64         return "Complaint"
65     else:
66         return "Feedback"
67 test_queries = [
68     "Where can I download the invoice?",
69     "Please refund my money",
70     "Your delivery is terrible",
71     "The chatbot is very helpful"
72 ]
73 for q in test_queries:
74     print("Predicted Type:", classify_few_shot(q))
75     print()
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-04/4.py"
● Few-shot Examples:
1. What are your working hours? -> Informational
2. Cancel my booking -> Transactional
3. My package is damaged -> Complaint
4. I like this app -> Feedback
Query: Where can I download the invoice?
Predicted Type: Informational

Few-shot Examples:
1. What are your working hours? -> Informational
2. Cancel my booking -> Transactional
3. My package is damaged -> Complaint
4. I like this app -> Feedback
Query: Please refund my money
Predicted Type: Transactional

Few-shot Examples:
1. What are your working hours? -> Informational
2. Cancel my booking -> Transactional
3. My package is damaged -> Complaint
4. I like this app -> Feedback
Query: Your delivery is terrible
Predicted Type: Complaint

Few-shot Examples:
1. What are your working hours? -> Informational
2. Cancel my booking -> Transactional
3. My package is damaged -> Complaint
4. I like this app -> Feedback
Query: The chatbot is very helpful
Predicted Type: Feedback
```

Comparsion:

Method	Correctness	Ambiguity Handling
Zero-Shot	Medium	Weak , no examples to guide intent
One-Shot	Good	Moderate , only one category learned
Few-Shot	Very High	Strong , all categories clearly shown

Observations:

♦ Zero-Shot Method

The model classifies queries only based on instructions and keywords.

It correctly identifies basic informational and transactional questions.

It may misclassify when a complaint is written politely (for example: “please help, order not delivered”).

No prior examples means more chances of ambiguity.

♦ One-Shot Method

One example helps the model understand what a “Complaint” looks like.

Classification accuracy improves compared to zero-shot.

Still limited because only one category is clearly demonstrated.

Ambiguous queries may still confuse between Transactional and Complaint.

♦ Few-Shot Method (Multiple Examples)

Multiple examples clearly define all four categories.

Model shows better consistency and fewer misclassifications.

Handles emotional words and action requests more accurately.

Best performance among the three methods.

Justification:

1. Zero-shot learning depends only on general language understanding, so it may confuse similar intents like refund vs complaint.
2. One-shot learning improves prediction by giving one labeled pattern, helping the model follow structure.
3. Few-shot learning provides clearer intent boundaries by showing multiple labeled examples.
4. More examples reduce uncertainty and improve classification confidence.
5. Therefore, few-shot prompting is more reliable for chatbot intent detection tasks.

5. Emotion Detection in Text

Scenario:

A mental-health chatbot needs to detect emotions: Happy, Sad, Angry, Anxious, Neutral.

Tasks:

1. Create labeled emotion samples.
2. Use Zero-shot prompting to identify emotions.
3. Use One-shot prompting with an example.
4. Use Few-shot prompting with multiple emotions.
5. Discuss ambiguity handling across techniques.

Zero Shot Prompt:

Generate a python code that Identify the emotion in the following text.Choose only one: Happy, Sad, Angry, Anxious, Neutral. Text: "{user_input}" Output only the emotion.Zero-Shot Emotion Detection (with user input)

Code:

```
5.py > ...
1  #Generate a python code that Identify the emotion in the following text.
2  #Choose only one: Happy, Sad, Angry, Anxious, Neutral.
3  #Text: "{user_input}"
4  #Output only the emotion.
5  #Zero-Shot Emotion Detection (with user input)
6
7  def emotion_zero_shot(text):
8      t = text.lower()
9
10     if any(w in t for w in ["happy", "excited", "great", "joy", "good"]):
11         return "Happy"
12     elif any(w in t for w in ["sad", "lonely", "empty", "cry", "tired"]):
13         return "Sad"
14     elif any(w in t for w in ["angry", "mad", "furious", "hate"]):
15         return "Angry"
16     elif any(w in t for w in ["worried", "scared", "nervous", "anxious", "fear"]):
17         return "Anxious"
18     else:
19         return "Neutral"
20
21     user_text = input("Enter your feeling: ")
22     print("Detected Emotion:", emotion_zero_shot(user_text))
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:\AI Assisted Coding\LAB\LAB-04\5.py"
Enter your feeling: mad
Detected Emotion: Angry
PS C:\AI Assisted Coding\LAB\LAB-04> 
```

One Shot Prompt:

Generate a python program that Identify the emotion in the text. Emotions: Happy, Sad, Angry, Anxious, Neutral.

Example:

Text: "I feel very lonely today."

Emotion: Sad

One-Shot Emotion Detection (no user input)

Code:

```
5.py > emotion_one_shot
25 #Generate a python program that Identify the emotion in the text.
26 #Emotions: Happy, Sad, Angry, Anxious, Neutral.
27 #Example:
28 #Text: "I feel very lonely today."
29 #Emotion: Sad
30 # One-Shot Emotion Detection (no user input)
31
32 def emotion_one_shot(text):
33     t = text.lower()
34
35     if any(w in t for w in ["lonely", "sad", "empty", "tired"]):
36         return "Sad"
37     elif any(w in t for w in ["happy", "excited", "great"]):
38         return "Happy"
39     elif any(w in t for w in ["angry", "mad", "furious"]):
40         return "Angry"
41     elif any(w in t for w in ["worried", "scared", "nervous"]):
42         return "Anxious"
43     else:
44         return "Neutral"
45
46 test_texts = [
47     "I am really excited about tomorrow",
48     "I feel empty and tired of everything",
49     "This situation makes me so mad",
50     "I am scared about my results",
51     "Just doing my daily routine"
52 ]
53 print("One-shot Example: 'I feel very lonely today' -> Sad\n")
54 for t in test_texts:
55     print("Text:", t)
56     print("Detected Emotion:", emotion_one_shot(t))
57     print()
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-04/5.py"
● One-shot Example: 'I feel very lonely today' -> Sad

Text: I am really excited about tomorrow
Detected Emotion: Happy

Text: I feel empty and tired of everything
Detected Emotion: Sad

Text: This situation makes me so mad
Detected Emotion: Angry

Text: I am scared about my results
Detected Emotion: Anxious

Text: Just doing my daily routine
Detected Emotion: Neutral

○ PS C:\AI Assisted Coding\LAB\LAB-04> |
```

Few Shot Prompt:

Generate a python program that Identify the emotion in the text using these categories: Happy, Sad, Angry, Anxious, Neutral.

Examples:

Text: "Today feels calm and normal."

Emotion:

Few-Shot Emotion Detection (no user input)

Code:

```
5.py > ...
58 #Generate a python program that Identify the emotion in the text using these categories:
59 #Happy, Sad, Angry, Anxious, Neutral.
60 #Examples:
61 #Text: "Today feels calm and normal."
62 #Emotion:
63 #Few-Shot Emotion Detection (no user input)
64 def emotion_few_shot(text):
65     t = text.lower()
66
67     if any(w in t for w in ["happy", "excited", "promoted", "great"]):
68         return "Happy"
69     elif any(w in t for w in ["sad", "nothing", "empty", "lonely"]):
70         return "Sad"
71     elif any(w in t for w in ["angry", "furious", "mad"]):
72         return "Angry"
73     elif any(w in t for w in ["worried", "worry", "scared", "nervous"]):
74         return "Anxious"
75     else:
76         return "Neutral"
77 test_texts = [
78     "I am really excited about tomorrow",
79     "I feel empty and tired of everything",
80     "This situation makes me so mad",
81     "I am scared about my future",
82     "Just doing my daily routine"
83 ]
84 print("Few-shot Examples:")
85 print("1. I got promoted today! -> Happy")
86 print("2. Nothing feels right anymore -> Sad")
87 print("3. This delay is making me furious -> Angry")
88 print("4. I keep worrying about tomorrow -> Anxious\n")
89 for t in test_texts:
90     print("Text:", t)
91     print("Detected Emotion:", emotion_few_shot(t))
92     print()
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-04> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-04/5.py"
● Few-shot Examples:
1. I got promoted today! -> Happy
2. Nothing feels right anymore -> Sad
3. This delay is making me furious -> Angry
4. I keep worrying about tomorrow -> Anxious

Text: I am really excited about tomorrow
Detected Emotion: Happy

Text: I feel empty and tired of everything
Detected Emotion: Sad

Text: This situation makes me so mad
Detected Emotion: Angry

Text: I am scared about my future
Detected Emotion: Anxious

Text: Just doing my daily routine
Detected Emotion: Neutral

○ PS C:\AI Assisted Coding\LAB\LAB-04> █
```

Ambiguity handling across techniques:

♦ **Zero-Shot Emotion Detection**

Zero-shot relies only on keyword matching and general understanding.

If a sentence contains mixed feelings (example: “I am tired but hopeful”), the model may misclassify.

Similar emotions like Sad and Anxious can be confused because no examples are given.

Ambiguity handling is weak due to lack of emotional context.

♦ **One-Shot Emotion Detection**

One example helps the model understand at least one emotional pattern clearly.

It reduces confusion for emotions similar to the example (like sadness).

However, emotions not demonstrated in the example may still be misclassified

Ambiguity handling is better than zero-shot but still limited.

♦ **Few-Shot Emotion Detection**

Multiple examples define boundaries between emotions clearly.

Similar emotions like Sad vs Anxious and Angry vs Sad are better distinguished.

The model can generalize emotional patterns more accurately.

Ambiguity is handled best in few-shot because emotional context is clearer.

Justification:

Zero-shot has no emotional references, so it depends only on keywords and may confuse similar emotions.

One-shot improves understanding by giving one emotional example, reducing some misclassification.

Few-shot provides multiple emotion patterns, allowing better separation of emotional categories.

More examples help the model learn emotional tone instead of just keywords.

Therefore, few-shot prompting is more reliable and safer for mental-health chatbots.